FarmConnect Diagrams (PlantUML Code)

```
' --- class-diagram.puml ---
@startuml
skinparam classAttributeIconSize 0

enum Role {
  PRODUCER
  BUYER
  ADMIN
}

enum UserStatus {
  ACTIVE
  SUSPENDED
  PENDING_VERIFICATION
  DELETED
}

enum VerificationStatus {
  UNVERIFIED
  PENDING
  APPROVED
  REJECTED
}

enum ProductStatus {
  DRAFT
  ACTIVE
  INACTIVE
  OUT_OF_STOCK
  ARCHIVED
}

enum OrderStatus {
  PENDING
  CONFIRMED
  PROCESSING
  SHIPPED
  DELIVERED
  CANCELLED
  REFUNDED
}

enum PaymentStatus {
  UNPAID
  PENDING
  PAID
  FAILED
  REFUNDED
  PARTIALLY_REFUNDED
}

enum DeliveryStatus {
  NOT_SHIPPED
  PREPARING
  IN_TRANSIT
  OUT_FOR_DELIVERY
  DELIVERED
  FAILED_DELIVERY
  RETURNED
}

enum DeliveryMethod {
  PICKUP
  DELIVERY
}

enum BuyerBusinessType {
  INDIVIDUAL
  RESTAURANT
  HOTEL
  WHOLESALE
  RETAIL
  OTHER
}
```

FarmConnect Diagrams (PlantUML Code)

```
enum PaymentMethod {
  CASH_ON_DELIVERY
  BANK_TRANSFER
  CIB_CARD
  EDAHABIA
}

enum PaymentEventType {
  INITIATED
  AUTHORIZED
  CAPTURED
  FAILED
  REFUNDED
  CHARGEBACK
}

enum PayoutStatus {
  PENDING
  PROCESSING
  PAID
  FAILED
}

enum NotificationType {
  ORDER_PLACED
  ORDER_CONFIRMED
  ORDER_SHIPPED
  ORDER_DELIVERED
  ORDER_CANCELLED
  PAYMENT_SUCCESS
  PAYMENT_FAILED
  PRODUCT_LOW_STOCK
  PRODUCER_VERIFIED
  PRODUCER_REJECTED
  GENERAL
}

enum NotificationChannel {
  IN_APP
  EMAIL
  SMS
  PUSH
}

enum AuditAction {
  CREATE
  UPDATE
  DELETE
  LOGIN
  LOGOUT
  VERIFY
  REJECT
  SUSPEND
  UNSUSPEND
}

enum CouponType {
  PERCENT
  FIXED
}

enum ReportTargetType {
  USER
  PRODUCT
  ORDER
}

enum ReportReason {
  SPAM
  FRAUD
  ABUSE
  INAPPROPRIATE_CONTENT
  OTHER
}
```

FarmConnect Diagrams (PlantUML Code)

```
enum ReportStatus {
  OPEN
  REVIEWING
  RESOLVED
  REJECTED
}

enum ProductUnit {
  KG
  PIECE
  BOX
}

enum ProductLogType {
  WATERING
  HARVEST
  FERTILIZE
  OTHER
}

class User {
  id: String
  email: String
  passwordHash: String
  fullName: String
  role: Role
  status: UserStatus
  avatarUrl: String?
  phone: String?
  createdAt: DateTime
  updatedAt: DateTime
  deletedAt: DateTime?
  refreshTokens: RefreshToken[]
  producerProfile: ProducerProfile?
  buyerProfile: BuyerProfile?
  addresses: Address[]
  cart: Cart?
  orders: Order[]
  reviews: Review[]
  favoriteProducts: FavoriteProduct[]
  favoriteProducers: FavoriteProducer[]
  reportsSubmitted: Report[]
  verificationReviews: ProducerVerificationRequest[]
  notifications: Notification[]
  aiRecommendations: AiRecommendation[]
  auditLogs: AuditLog[]
}

class RefreshToken {
  id: String
  token: String
  userId: String
  expiresAt: DateTime
  revokedAt: DateTime?
  userAgent: String?
  ipAddress: String?
  createdAt: DateTime
  user: User
}

class ProducerProfile {
  id: String
  userId: String
  businessName: String
  businessType: String?
  bio: String?
  latitude: Decimal?
  longitude: Decimal?
  producerOffersDelivery: Boolean
  wilaya: String
  commune: String
  nif: String?
  nis: String?
  nifDocumentUrl: String?
  verificationStatus: VerificationStatus
```

FarmConnect Diagrams (PlantUML Code)

```
  verifiedAt: DateTime?
  verifiedById: String?
  createdAt: DateTime
  updatedAt: DateTime
  user: User
  products: Product[]
  verificationRequests: ProducerVerificationRequest[]
  payouts: Payout[]
  reviewsReceived: Review[]
  favoritedBy: FavoriteProducer[]
  coupons: Coupon[]
}

class ProducerVerificationRequest {
  id: String
  producerProfileId: String
  status: VerificationStatus
  notes: String?
  documents: Json
  submittedAt: DateTime
  reviewedAt: DateTime?
  reviewedById: String?
  producerProfile: ProducerProfile
  reviewer: User?
}

class BuyerProfile {
  id: String
  userId: String
  businessType: BuyerBusinessType?
  createdAt: DateTime
  updatedAt: DateTime
  user: User
}

class Address {
  id: String
  userId: String
  label: String
  recipientName: String
  phone: String
  wilaya: String
  commune: String
  street: String
  postalCode: String?
  isDefault: Boolean
  createdAt: DateTime
  updatedAt: DateTime
  user: User
  orders: Order[]
}

class Category {
  id: String
  nameEn: String
  nameAr: String
  slug: String
  parentId: String?
  createdAt: DateTime
  updatedAt: DateTime
  parent: Category?
  children: Category[]
  products: Product[]
  aiForecasts: AiForecast[]
}

class Product {
  id: String
  producerId: String
  categoryId: String?
}

class ProductLog {
  id: String
  productId: String
```

FarmConnect Diagrams (PlantUML Code)

```
  type: ProductLogType
  note: String
  happenedAt: DateTime
  createdBy: String
  createdAt: DateTime
  product: Product
}

class ProductImage {
  id: String
  productId: String
  url: String
  altText: String?
  position: Int
  createdAt: DateTime
  product: Product
}

class Cart {
  id: String
  buyerId: String
  createdAt: DateTime
  updatedAt: DateTime
  buyer: User
  items: CartItem[]
}

class CartItem {
  id: String
  cartId: String
  productId: String
  quantity: Int
  createdAt: DateTime
  updatedAt: DateTime
  cart: Cart
  product: Product
}

class Order {
  id: String
  buyerId: String
  buyerAddressId: String?
  status: OrderStatus
  paymentStatus: PaymentStatus
  deliveryStatus: DeliveryStatus
  deliveryMethod: DeliveryMethod
  deliveryFee: Decimal?
  deliveryVerificationToken: String?
  verifiedAt: DateTime?
  couponCode: String?
  discountAmount: Decimal?
  total: Decimal
  currency: String
  notes: String?
  createdAt: DateTime
  updatedAt: DateTime
  buyer: User
  buyerAddress: Address?
  items: OrderItem[]
  payment: Payment?
  deliveryTrackings: DeliveryTracking[]
  reviews: Review[]
}

class OrderItem {
  id: String
  orderId: String
  productId: String?
}

class Review {
  id: String
  orderId: String
  productId: String
  producerId: String
```

FarmConnect Diagrams (PlantUML Code)

```
  buyerId: String
  rating: Int
  comment: String?
  createdAt: DateTime
  order: Order
  product: Product
  producer: ProducerProfile
  buyer: User
}

class FavoriteProduct {
  id: String
  buyerId: String
  productId: String
  createdAt: DateTime
  buyer: User
  product: Product
}

class FavoriteProducer {
  id: String
  buyerId: String
  producerId: String
  createdAt: DateTime
  buyer: User
  producer: ProducerProfile
}

class Coupon {
  id: String
  code: String
  producerId: String
  type: CouponType
  amount: Decimal
  startsAt: DateTime
  endsAt: DateTime
  usageLimit: Int?
  usedCount: Int
  isActive: Boolean
  createdAt: DateTime
  updatedAt: DateTime
  producer: ProducerProfile
}

class Report {
  id: String
  reporterId: String
  targetType: ReportTargetType
  targetId: String
  reason: ReportReason
  description: String
  status: ReportStatus
  internalNote: String?
  createdAt: DateTime
  updatedAt: DateTime
  reporter: User
}

class Payment {
  id: String
  orderId: String
  method: PaymentMethod
  status: PaymentStatus
  amount: Decimal
  currency: String
  gatewayRef: String?
  gatewayResponse: Json?
  paidAt: DateTime?
  failedAt: DateTime?
  createdAt: DateTime
  updatedAt: DateTime
  order: Order
  events: PaymentEvent[]
}
```

FarmConnect Diagrams (PlantUML Code)

```
class PaymentEvent {
  id: String
  paymentId: String
  type: PaymentEventType
  payload: Json
}

class Payout {
  id: String
  producerProfileId: String
  amount: Decimal
  currency: String
  status: PayoutStatus
  bankName: String?
  bankAccount: String?
  reference: String?
  notes: String?
  processedAt: DateTime?
  createdAt: DateTime
  updatedAt: DateTime
  producer: ProducerProfile
}

class DeliveryTracking {
  id: String
  orderId: String
  status: DeliveryStatus
  location: String?
  description: String?
  occurredAt: DateTime
  order: Order
}

class Notification {
  id: String
  userId: String
  type: NotificationType
  channel: NotificationChannel
  title: String
  body: String
}

class AnalyticsDaily {
  id: String
  date: DateTime
  wilaya: String?
  newUsers: Int
  newProducers: Int
  newBuyers: Int
  totalOrders: Int
  totalRevenue: Decimal
  createdAt: DateTime
  updatedAt: DateTime
}

class AiRecommendation {
  id: String
  userId: String
  productId: String
  score: Float
  reason: String?
  seenAt: DateTime?
  clickedAt: DateTime?
  createdAt: DateTime
  user: User
  product: Product
}

class AiForecast {
  id: String
  productId: String?
  categoryId: String?
  forecastDate: DateTime
  predictedDemand: Decimal
  confidenceScore: Float
```

FarmConnect Diagrams (PlantUML Code)

```
  modelVersion: String
  createdAt: DateTime
  product: Product?
  category: Category?
}

class AuditLog {
  id: String
  actorId: String?
  targetType: String
  targetId: String
  action: AuditAction
}

Address --> User : user
AiForecast --> Category : category
AiForecast --> Product : product
AiRecommendation --> Product : product
AiRecommendation --> User : user
BuyerProfile --> User : user
Cart --> User : buyer
CartItem --> Cart : cart
CartItem --> Product : product
Category --> Category : parent
Coupon --> ProducerProfile : producer
DeliveryTracking --> Order : order
FavoriteProducer --> ProducerProfile : producer
FavoriteProducer --> User : buyer
FavoriteProduct --> Product : product
FavoriteProduct --> User : buyer
Order --> Address : buyerAddress
Order --> Payment : payment
Order --> User : buyer
Payment --> Order : order
Payout --> ProducerProfile : producer
ProducerProfile --> User : user
ProducerVerificationRequest --> ProducerProfile : producerProfile
ProducerVerificationRequest --> User : reviewer
ProductImage --> Product : product
ProductLog --> Product : product
RefreshToken --> User : user
Report --> User : reporter
Review --> Order : order
Review --> ProducerProfile : producer
Review --> Product : product
Review --> User : buyer
User --> BuyerProfile : buyerProfile
User --> Cart : cart
User --> ProducerProfile : producerProfile

@enduml

' --- use-cases.puml ---
@startuml
left to right direction

actor Buyer
actor Producer
actor Admin
actor "Payment Gateway" as PG
actor "Delivery Partner" as DP

rectangle "FarmConnect" {
  (Register / Login) as UC_Login
  (Manage Profile) as UC_Profile
  (Browse Products) as UC_Browse
  (Manage Cart) as UC_Cart
  (Checkout) as UC_Checkout
  (Apply Coupon) as UC_Coupon
  (Pay Online) as UC_PayOnline
  (Cash on Delivery) as UC_COD
  (Track Order) as UC_Track
  (Leave Review) as UC_Review
  (Favorite Product) as UC_FavProduct
  (Favorite Producer) as UC_FavProducer
```

FarmConnect Diagrams (PlantUML Code)

```
  (Report Issue) as UC_Report

  (Manage Products) as UC_Products
  (Manage Orders) as UC_Orders
  (Update Delivery Status) as UC_Delivery
  (Request Payout) as UC_Payout
  (Verify Delivery QR) as UC_QR

  (Verify Producer) as UC_VerifyProducer
  (Moderate Reports) as UC_Moderate
  (View Audit Logs) as UC_Audit
}

Buyer --> UC_Login
Buyer --> UC_Profile
Buyer --> UC_Browse
Buyer --> UC_Cart
Buyer --> UC_Checkout
Buyer --> UC_Track
Buyer --> UC_Review
Buyer --> UC_FavProduct
Buyer --> UC_FavProducer
Buyer --> UC_Report

Producer --> UC_Products
Producer --> UC_Orders
Producer --> UC_Delivery
Producer --> UC_Payout
Producer --> UC_QR

Admin --> UC_VerifyProducer
Admin --> UC_Moderate
Admin --> UC_Audit

UC_Checkout .> UC_Coupon : <<include>>
UC_Checkout .> UC_PayOnline : <<extend>>
UC_Checkout .> UC_COD : <<extend>>

PG --> UC_PayOnline
DP --> UC_Delivery

@enduml

' --- sequence-checkout.puml ---
@startuml
title Place Order (Cash on Delivery)

actor Buyer
participant "Web App" as Web
participant "API" as API
database "PostgreSQL" as DB
participant "Notification" as Notif

Buyer -> Web: Proceed to checkout
Web -> API: POST /orders (cart, address, coupon?)
API -> DB: Validate stock, price, coupon
API -> DB: Create Order + OrderItems
API -> DB: Create Payment (UNPAID)
API -> DB: Create DeliveryVerificationToken
API -> Notif: Notify producer + buyer
API --> Web: Order created (status=PENDING)
Web --> Buyer: Show confirmation

alt Coupon invalid
  API --> Web: 400 invalid coupon
  Web --> Buyer: Show error
end

@enduml

' --- sequence-payment.puml ---
@startuml
title Online Payment (Bank/CIB/Edahabia)

actor Buyer
```

FarmConnect Diagrams (PlantUML Code)

```
participant "Web App" as Web
participant "API" as API
database "PostgreSQL" as DB
participant "Payment Gateway" as PG
participant "Notification" as Notif

Buyer -> Web: Pay online
Web -> API: POST /payments (orderId, method)
API -> DB: Create Payment (PENDING)
API -> PG: Initiate payment
PG --> API: Payment authorized
API -> DB: Record PaymentEvent (AUTHORIZED)
API -> PG: Capture payment
PG --> API: Payment captured
API -> DB: Update Payment (PAID)
API -> DB: Update Order (CONFIRMED)
API -> DB: Record PaymentEvent (CAPTURED)
API -> Notif: Notify buyer + producer
API --> Web: Payment success
Web --> Buyer: Show receipt

alt Payment failed
  PG --> API: Failure
  API -> DB: Update Payment (FAILED)
  API -> DB: Record PaymentEvent (FAILED)
  API -> Notif: Notify buyer
  API --> Web: Payment failed
  Web --> Buyer: Show error
end

@enduml

' --- sequence-delivery-verification.puml ---
@startuml
title Delivery QR Verification

actor "Delivery Partner" as DP
participant "Mobile/Web" as App
participant "API" as API
database "PostgreSQL" as DB
participant "Notification" as Notif

DP -> App: Scan QR / enter token
App -> API: POST /deliveries/verify (token)
API -> DB: Find order by token
API -> DB: Update DeliveryStatus=DELIVERED
API -> DB: Set verifiedAt
API -> Notif: Notify buyer + producer
API --> App: Verification success

alt Token invalid or expired
  API --> App: 400 invalid token
end

@enduml
```